

GAMEVAULT

Development Roadmap

A dependency-led plan for one connected game library.

DEVELOPMENT PHILOSOPHY

A structured development plan for expanding GameVault from its current PC application into a more complete game-library, tracking, discovery, achievement, pricing, and cross-platform ecosystem.

The roadmap should be approached in dependency order rather than simply implementing the most visible features first.

The recommended development sequence is:

Foundation → Library Connections → Library Data → Play Tracking → Collection UI → Artwork → Achievements → Pricing → Recommendations → Wishlist & Notifications → Onboarding → Mobile

This allows the underlying systems to mature before more complex features depend on them and should significantly reduce the amount of functionality that needs to be rebuilt later.

PHASE HEADINGS INCLUDE FILLABLE DONE CHECKBOXES. SAVE THE PDF AFTER EDITING.

UPDATED 05 OCTOBER 2026 / ORIGINAL ROADMAP + ALL RESEARCH INCORPORATED

11

PHASES

48

NUMBERED ITEMS

41

RESEARCH TOPICS

One library. Every game. Your entire gaming history.

Plan, scope and phase navigation

The owner explicitly promoted all research in this revision. Keep the original 11 phases, items 1-35, additions 36-48, rationale and final path. Research IDs R01-R41 identify the added work without renumbering existing items. All are plans or evidence-gated investigations, not app implementation. Future research is not automatically promoted. Conditional studies remain conditional; paid services and reporting destinations still need the owner's decisions.

Latest public release rechecked: 0.3.9, published 5 October 2026 at 13:26 Europe/Oslo [77, 90]. Existing protected storage, signed updater, recovery/task visibility, Steam achievements and store search are foundations. Earlier 0.3.8 and pre-release code-study observations stay labelled historical; no new app tests or performance gains are claimed. R18's old guide-version mismatch was corrected in 0.3.9; retain release consistency checks.

01	Architecture & Platform Foundation	03
02	Library Connection System	08
03	Library Data & Play Tracking	10
04	Collection Experience	12
05	Cover & Artwork System	14
06	Achievements	15
07	Pricing & Library Value	16
08	Recommendations & Discovery	17
09	Wishlist & Online Notifications	18
10	First-Time Setup Experience	19
11	Mobile GameVault	20

Current direction: Current clarifications override contradictory historical wording: Settings opens inside the app; SteamGridDB/custom uploads remain the cover direction; local/manual onboarding is optional; Comfort is a modifier of exactly five themes. Read the notes adjacent to affected items.

Completion: Tick Done only after the phase's original items, attached additions/research requirements and applicable cross-phase checks pass, with completed version/date and evidence recorded. Conditional investigations require a documented result and agreed scope; an unsupported dependency needs an explicit fallback or accepted deferral. This document update does not finish a development phase. Preserve saved checkbox values when rebuilding.

PHASE 01 / ITEMS 01-03

Architecture & Platform Foundation

DONE ☐

01 Widescreen & Ultrawidescreen Support

- Add proper support for widescreen and ultrawidescreen displays.
- Make the existing interface responsive across different resolutions and aspect ratios.
- Establish reusable layout rules that future GameVault screens and components automatically inherit.

02 Prepare GameVault for Online Functionality

- Determine which functionality should remain local and which functionality requires online services.
- Establish the architecture required for:
 - Email notifications
 - Wishlist price monitoring
 - Cross-device synchronization
 - Remote account functionality
 - Future mobile integration
- Preserve offline functionality wherever practical.

03 Establish Platform Separation

GameVault's current desktop version should primarily be treated as the **PC version**. For **What to Play**, only PC games should be displayed. For general platform sorting, retain:

- PC
- Mobile
- All
- Other

The underlying data structure should already anticipate the future mobile application so the two applications can eventually communicate without requiring a major architectural rewrite.

36 Optional GameVault Account & Remember Me

Phase 1: account architecture. Phase 10: sign-up and login experience.

Offer the option to create and log in to a GameVault account. Preserve practical local/offline use without requiring an account for every feature.

- Include a checkable Remember me option with choices of 7 days, 14 days, 30 days, or 6 months.
- Remember the device for the selected period so routine use does not repeatedly require login. Treat this as a maximum persistence window; logout, revoked access, expiry, or suspicious activity can end it earlier.
- Show a clear warning when 6 months is selected. Proposed wording: For security, we recommend refreshing your login at least every 30 days. Only choose 6 months on a private, trusted device.
- Store protected, revocable login tokens rather than a saved plaintext password. Require fresh authentication for sensitive changes such as password, email, or two-factor settings.

The 30-day reminder is a proposed product choice, not a security-standard certification. Reference: OWASP Session Management.

37 Optional Two-Factor / Two-Step Verification

Phase 1: security design and service choice. Phase 10: enrollment and recovery UI.

Let account users enable additional sign-in verification through an authenticator app, a verified mobile number using SMS, or email verification codes.

- Recommend an authenticator app; explain the different protection levels of the available methods.
- Keep recovery and the loss of a phone/authenticator in scope when designing enrollment.
- Treat email as a weaker two-step option: NIST excludes email as an out-of-band authenticator and restricts telephone-network authentication. Do not label all three methods as equivalent strong MFA.
- SMS/email service selection remains subject to the owner's payment rule.

Reference: NIST SP 800-63B.

38 Recurring Code Review & Maintenance

All phases: review after major milestones and before each public release.

Regularly review the code for optimizations, cleanup, fixes, duplication, outdated dependencies, and maintainability. Measure performance before and after optimizations; prioritize demonstrated problems. Record findings and verify that cleanup preserves expected behavior. Include security and dependency checks in release preparation. The exact review cadence can be agreed as development progresses.

40 Mandatory Owner Decision Before Any Cost

Project-wide rule: applies immediately and throughout every phase.

If any planned action requires the owner to pay, work on that action must stop until the owner personally decides. The assistant must never make that financial choice on the owner's behalf.

- Explain the purpose, provider, price, billing frequency, likely usage charges, and available alternatives before requesting the decision.
- Do not purchase, subscribe, activate billable usage, accept a paid upgrade, or enroll in a trial that creates a payment commitment without explicit owner authorization.
- If pricing, limits, or future charges are unclear, stop the affected action and resolve the uncertainty before proceeding.
- A prior approval for one expense does not authorize a different expense or increased scope.

This rule covers hosting, accounts, SMS/email delivery, domains, APIs, signing, reporting services, and any other paid dependency. Continue independent work that has no cost commitment.

43 Phase Completion Tracking

All phases: a fillable Done checkbox appears in every phase heading.

The owner or assistant can tick a phase once its agreed work has been successfully finished and verified. Completion means its required items and applicable additional requirements work as intended, the relevant checks pass, and all phase-blocking issues are resolved. Record the completed version/date and validation evidence in the working project record. The assistant should update the saved roadmap when a phase is verified as complete. Save the PDF after editing its checkboxes to retain their state. All phases begin unchecked in this revision.

44 Open Settings as an In-App Panel

Phase 1: interface foundation. Phase 4: collection experience.

Clicking the cogwheel should open a window-style Settings panel inside the existing GameVault window, rather than launching a separate application window. This supersedes the earlier separate-window Settings design.

- Keep the main GameVault window and collection context in place. Give the panel a clear Close button and keyboard access; closing it should return focus to the cogwheel.
- Preserve unfinished settings edits while background status changes arrive. For unsaved edits, provide a clear save/discard choice when needed.
- Reopening Settings must show the current connection and sync state without restarting a task.
- Fit the panel to small windows and enlarged text while keeping its controls reachable.

46 Regular Workflow and Interruption Regression Checks

All phases: relevant checks after changes, full workflow checks at milestones and before releases.

Maintain a recurring regression checklist specifically for tasks interrupted by normal user actions. Any confirmed bug should become a repeatable regression check before its fix is considered complete. Use the research PDF's testing matrix to expand the coverage.

- Test closing/reopening Settings, changing screens, minimizing the app and resizing during library, artwork, achievement and wishlist work.
- Test explicit cancellation, repeated clicks, concurrent tasks, slow or failed providers, late responses, failed saves, app restart and sleep/resume.
- Verify that games, notes, artwork, sessions, notification rules and progress stay correct, with no silent cancellation, duplicate results or loss of completed work.
- Check keyboard/focus behavior, translated controls, offline use, backups/restoration, large collections and installed Windows behavior.
- Run focused checks after affected changes, the broader workflow suite after major milestones, and installed-app/release checks before publication. Keep first-run failures and intermittent failures visible rather than hiding them behind retries.
- Use isolated sample data for failure tests and separate hands-on checks for real provider accounts; never create paid requests without the owner's decision. Record version, test conditions, outcome and any remaining limitations.

47 Five Distinct Themes and Settings-Based Selection

Phase 1: appearance foundation. Phase 10: final interface polish.

Provide exactly five carefully designed base themes, with the theme dropdown kept inside Settings > Appearance. Existing palettes may be refined or replaced when a better option serves users.

- Recommended set: Gaming (navy/cyan default), Dark (neutral charcoal), Light (neutral light), Paper (warm light), and Calm (muted green/gray). Paper is a proposed replacement for Cozy, not an extra sixth theme.
- Polish the default with consistent surfaces, readable typography, restrained accents and clear focus/status states. Keep covers in their original colors.
- Include a live preview and preserve selection across restart and backup/restore. If retiring Cozy, agree a migration that preserves old preferences until the user confirms a replacement.
- Use research on comparable apps and accessibility to guide the choices. Do not label any palette most used without reliable usage evidence. The five-theme recommendation and sample colors are proposals, not implemented app changes.

Completion check: all five themes work across the app, selection stays inside Settings, previous preferences migrate safely, and contrast, keyboard access and enlarged text checks pass.

48 Optional Comfort Subtheme for All Five Themes

Phase 1: shared appearance rules. Phase 10: comfort and accessibility validation.

Add a checkable Comfort mode option below the Settings theme dropdown. This is a modifier of the selected theme, not a sixth base theme. Adapt it individually to Gaming, Dark, Light, Paper and Calm.

- Retain each theme's character and light/dark polarity; soften large surfaces and accents, remove decorative glow and unnecessary motion, and keep text, control boundaries, focus and errors clearly readable. Do not dim or recolor all artwork.
- Keep Light and Paper light. Tailor Gaming to softer navy/cyan, Dark to warmer charcoal, Light to neutral off-white, Paper to less intense ivory/ochre, and Calm to softer sage with visible panel separation.
- Maintain at least 4.5:1 normal text contrast, 3:1 large text and required non-text contrast; respect OS reduced motion even when the checkbox is off and preserve Windows contrast/user preferences.
- Save Comfort independently of the theme; keep it checked when switching themes. Off restores the selected base palette. Restart and backup/restore must preserve both settings; old files without Comfort default to off.
- Present this as a visual-comfort preference. Research does not establish that these palettes prevent eye damage or reliably improve sleep. Optional help can explain suitable brightness, glare, breaks and evening screen habits.
- Test every theme with Comfort off and on (10 combinations), including real screens/states, scaling, old backups, failed preference saves and switching during edits or sync. No draft, job or saved result should be lost.

Completion check: the five tailored variants, persistent Settings checkbox, measured accessibility checks and workflow regressions pass. See the research PDF's theme study for evidence, palette examples and the evaluation plan.

R01 Backup history and tested restoration

Planned improvement | Applies to items 1, 31, 32, 38

Requirement: Keep dated rotating snapshots, expose backup age/location and offer a restore preview. Preserve a current snapshot before replacing data; define retention and disk limits. Back up before migrations and prove restoration on a second installation. Reconnect providers rather than restoring live credentials.

Rationale: The previous-save slot is replaced by later saves, so it may not cover an older mistake. Rotation improves recovery; it does not replace a copy on another device. Playnite and SQLite describe established backup approaches [1, 2].

Verify: Restore an older snapshot after several later saves; compare records and artwork, test corruption/full disk, and confirm credentials are absent.

R06 Optional accounts, recovery and trusted devices

Evidence-gated investigation | Applies to items 2, 36, 37

Requirement: Keep optional sign-in and the requested 7, 14, 30-day or 6-month persistence. Warn at 6 months and propose refreshing sign-in every 30 days; this interval is a product choice. Add device history, revocation, Sign out everywhere, recovery codes and lost-authenticator recovery. Evaluate passkeys; require fresh verification for sensitive actions.

Rationale: Long persistence must remain revocable. OWASP covers recovery and reauthentication [6, 7, 9]. Preserve authenticator/SMS/email choices but explain their different strength; email verification or recovery is not NIST-equivalent out-of-band authentication [8].

Verify: Recover a lost factor, expire/revoke a remembered device and confirm an account is unnecessary for the local collection.

R07 Privacy controls and account-data lifecycle

Planned improvement | Applies to items 2, 29, 33, 35, 39, 42

Requirement: Expand this into a concise privacy center: local/uploaded/public/provider data, optional analytics consent, diagnostic consent, export and account deletion. Define retention in synchronized records and backups, access limits and deletion timing. Keep any public gaming profile private by default.

Rationale: Hosting changes the privacy model even if today's app is local. European Commission guidance supports privacy by design and minimization [10]; assess applicable obligations before choosing markets or collecting account data.

Verify: Trace each uploaded field to a purpose and consent setting; demonstrate account deletion across active data and the documented backup lifecycle.

R12 Accessibility and display scaling across the app

Planned improvement | Applies to items 1, 14, 44, 47, 48

Requirement: Test the installed Tauri/WebView app with keyboard and Windows screen readers, 200% text size and high DPI. Check focus after redraws, modal focus/return, scrolling panels, error announcements, contrast and non-color status cues. Test ultrawide and small windows together; review translated advanced controls.

Rationale: Existing accessibility attributes are a useful foundation, but they do not prove usable workflows. Microsoft and WCAG identify keyboard, programmatic access, contrast and text-resizing requirements [16, 37].

Verify: Complete import, filtering, achievements, updates and reporting without a mouse or clipped controls at enlarged text size.

R13 Measured large-library performance and storage limits

Planned improvement | Applies to items 1, 5, 13, 15, 38

Requirement: Establish the benchmark protocol in the code-study page before selecting R23-R30. Measure startup, typing, scrolling, save stages, total WebView2/native memory and background CPU at realistic data sizes. Separate metadata scaling from artwork capacity; stress fixtures can exceed the current cap and are not supported-save promises. Set budgets from the measured baseline.

Rationale: Lazy image loading does not bound the number of rendered elements or whole-collection writes. CSS containment can reduce off-screen rendering [17], but choose improvements from the actual JavaScript/WebView measurements, with keyboard access intact.

Verify: Publish source/build, hardware, fixture bytes and repeated before/after timings. Keep the existing operation-count checks, add latency/memory coverage, and test near-cap failure/cancellation without data loss. The 500-game/20,000-session test asserts counts and correctness, not wall-clock latency [67].

R20 Regression checks for real user interruptions

Planned improvement | Applies to items 38, 43, 46

Requirement: Turn every confirmed bug into a focused regression. For the Settings issue, hold provider responses until the panel has closed, then verify committed results in the installed Windows app. Cover the matrix below using isolated data, controlled delays and failures, plus separate owner-account checks. Record exact build, conditions, first-run results and remaining gaps.

Rationale: Playwright supports network simulation and WebView2 automation [41, 42]; native requests also need injected fixtures or test providers. Tauri describes distinct test layers [43]. Retried passes should remain labeled intermittent [44], and automated accessibility checks do not replace manual interaction [45].

Verify: Pass the interruption matrix on the fixed installed build, retain an explanatory failure trace, and rerun the relevant cases after future lifecycle/sync changes.

R21 Five purposeful themes and a polished default

Planned improvement | Applies to items 47

Requirement: Recommend Gaming, neutral Dark, Light, Paper and Calm. Replace Cozy only through a safe preference migration; keep five choices and a preview in Settings. The theme study explains each role.

Rationale: Three dark and two light options broaden the current range. Comparable products support choice, but reviewed sources do not establish a most-used palette [49, 50, 51, 52].

Verify: Verify all screens, contrast, scaling, keyboard access, safe migration, saved selection and preserved drafts.

R22 Comfort as a tailored option on every theme

Planned improvement | Applies to items 48

Requirement: Add a checkbox below the Settings dropdown. Apply tailored palettes without changing light/dark polarity; preserve readable states and persist Comfort across theme changes. See the rules and previews below.

Rationale: Comfort is personal. Reading and sleep evidence support choice and cautious wording, not eye-protection or reliable sleep-benefit claims [53, 54, 55].

Verify: Test all ten combinations, old/new backups, failed saves, active sync, drafts and OS accessibility preferences.

R25 Reduce full-collection copying and repeated saves

Evidence-gated investigation | Applies to items 2, 5, 38, 45

Requirement: Reuse a validated snapshot flag and one serialized payload per save. Evaluate small, bounded background batches with explicit durable checkpoints and visible unsaved status. Keep user edits immediately durable. Later compare revision-based conflict checks and smaller record writes if the measured cost remains high.

Rationale: Repeated processing includes artwork and history even for a small preference change. Fewer passes/writes could help; blindly debouncing saves or retaining an old clone risks lost edits. Preserve transactional recovery and conflict checks [74].

Verify: Measure clone/validation/JSON/IPC/database stages separately. Interrupt batches, fail writes, edit during sync and save from two windows; retain committed work and detect stale writers.

R26 Keep database work off the native UI thread

Evidence-gated investigation | Applies to items 1, 2, 38, 45

Requirement: Evaluate a dedicated database worker with bounded requests and a reused connection; perform schema setup at initialization. Keep blocking SQLite/JSON work off the native UI thread. Merely adding async around blocking work is insufficient. Send acknowledgement only after commit.

Rationale: Tauri documents synchronous commands running on the main thread by default [70]. Larger saves or lock waits can therefore affect responsiveness; no freeze was reproduced here. SQLite allows one simultaneous writer [74], so uncontrolled parallel writes are unsuitable.

Verify: Measure input/window responsiveness during near-limit saves and forced lock waits. Verify conflict, recovery, full-disk, updater and orderly shutdown behavior. Compare connection setup cost [76].

R30 Make the source easier to review without a framework rewrite

Evidence-gated investigation | Applies to items 2, 38 | Sources [60, 68]

Requirement: Format maintained source and gradually extract explicit collection, persistence, task and view interfaces. Define command/result and collection contracts with checked JSDoc or TypeScript if useful. Preserve the generated-preview workflow; add a drift check. Reuse selectors instead of parallel copies of totals/search logic.

Rationale: Clear boundaries help spot stale state, unsafe saves and lifecycle bugs. Formatting/types do not themselves make the app faster. Existing modules and tests are a foundation; no evidence requires a new frontend framework or database replacement.

Verify: Keep behavior/recovery tests and equivalent browser/native flows. Remove code only after proving it has no callers; measure bundle/startup impact and avoid refactors that mix unrelated behavior changes.

Phase completion gate

Verify offline/local use, responsive accessible layout, protected saves/backups, shared task ownership and the measured performance baseline. Account/security/privacy designs, five themes plus all ten Comfort combinations and maintenance/test contracts are agreed. Conditional optimization experiments have measured results or a recorded deferral; no speedup is assumed.

PHASE 02 / ITEMS 04-07

Library Connection System

DONE ☐

04 Create a Unified "Connect Libraries" System

Merge the existing **Connect Steam** and **Connect Libraries** functionality into one unified **Connect Libraries** control. The control should provide access to supported services such as:

- Steam
- GOG
- Epic Games
- Future supported libraries

Selecting a service should open its dedicated configuration interface for API keys, authentication, or any other required information. Users should also be able to individually enable or disable libraries for importing. Once all libraries the user wants are configured, the interface should display: **Libraries Connected**

05 Standardize Library Synchronization

Create a unified internal representation for games imported from different services. The synchronization system should:

- Handle games from multiple launchers consistently.
- Recognize duplicate ownership across different platforms.
- Preserve platform-specific ownership information.
- Normalize imported metadata.
- Provide a reliable foundation for future cross-platform functionality.

06 Synchronize Favorites

Current clarification: Implement automatic favorites only for verified supported sources. Unsupported services keep manual favorites; a missing field must not clear the user's saved choice. R02/R33 refine this item.

Where supported by the external platform:

- Retrieve favorited games from connected libraries.
- Automatically mark those games as favorites inside GameVault.

07 Implement DLC Handling

Current clarification: Separate a DLC/base relationship from ownership and installation. Keep unknown or orphan DLC reviewable; do not claim complete cross-launcher DLC coverage before validating legitimate access. R04/R33 refine this item.

During synchronization:

- Detect DLC separately from base games.
- Determine the associated base game.
- Display DLC underneath or within its respective game rather than treating every DLC as an independent library title.

45 Keep Library and Artwork Sync Running After Settings Closes

Phase 2: library synchronization. Phase 5: artwork synchronization.

Once a library or artwork sync has started, closing Settings or moving to another GameVault screen must not stop it. Apply this to Steam and every other supported library connection.

- Give active tasks an app-wide lifetime, independent of the Settings panel. Keep a visible progress/status indicator outside Settings, with an explicit Cancel action.
- Preserve completed work, avoid duplicate jobs or imports, and show the same active progress when Settings is reopened.
- If an import still needs review or permission, retain the pending review and clearly identify that requirement instead of silently discarding it.
- Handle provider failures, interrupted network access and failed saves without losing existing games or artwork; retry unresolved work without repeating successful steps unnecessarily.
- Closing the entire application is a separate case: save safe progress and offer continuation after restart where practical, or explain what remains. Do not imply that work continues after the app exits.

Completion check: start a delayed sync from Settings, close the panel during both library and artwork stages, navigate elsewhere, then reopen it. Verify the task finishes once and all saved results remain after restarting GameVault.

R02 Connector capabilities and honest connection status

Planned improvement | Applies to items 4, 5, 6, 7, 8

Requirement: Add a compact per-service matrix for ownership, favorites, DLC, achievements, Last Played, purchase history, install detection and launch support. Show data source, last successful scan, privacy limits and specific failure states. Distinguish configured, authenticated and successfully synchronized.

Rationale: Users can otherwise interpret Connected as full account access. Steam ownership depends on visibility, and unsupported fields need explicit fallbacks [3]. Existing failure-preservation behavior should remain intact.

Verify: For every provider, test a private account or unsupported field, empty scan and outage without removing existing games.

R04 Repairable matches and protected metadata

Planned improvement | Applies to items 5, 7, 15, 16

Requirement: Offer a review queue for skipped matches, reversible merge/split, edition and base-game/DLC relationships, and per-field source/refresh details. Let users pin corrections. Preview replacement artwork and report unmatched, unavailable and failed results separately; keep SteamGridDB and custom uploads as the agreed choices.

Rationale: Conservative matching avoids many bad merges but leaves legitimate matches unresolved. Clear provenance makes corrections auditable, while pinned edits avoid repeated rework [4]. Removed AI and Steam-default cover providers should not return.

Verify: Correct an ambiguous edition, undo a merge and refresh metadata without losing notes, sessions, chosen artwork or contributor credit.

R19 In-app Settings and independent background work

Planned improvement | Applies to items 4, 5, 44, 45 | Sources [26, 40]

Requirement: Use an in-app Settings panel and move job ownership to the main app or native task service. Separate library import/review from artwork stages, with one active job per target, saved progress and explicit Cancel. Keep progress visible outside Settings; reopening should reconnect to the existing job. Retain pending import review and protect unfinished Settings edits.

Rationale: Task lifetime should follow the app, not the panel used to start it. Window-local orchestration makes closing the window a plausible lifecycle failure, not a confirmed root cause. A shared owner also prevents duplicate work and improves recovery after navigation or transient failure.

Verify: Close Settings during delayed library and artwork stages, navigate/reopen, then verify one completed result after restart. Treat full app exit separately; promise continuation only if recovery is implemented.

R28 Reuse network connections and avoid duplicate lookups

Evidence-gated investigation | Applies to items 4, 5, 18, 21, 38

Requirement: Reuse an application-managed Client with the existing configuration and per-request credentials. Coalesce identical concurrent lookups; cache stable public metadata/icons with bounded retention. Key prices by app, region and currency and private data by account. Respect provider rate limits; do not add unlimited parallel requests.

Rationale: request recommends reusing its Client because it contains a connection pool [69]. This is a well-supported mechanism, but network conditions determine gains. Cached prices and achievements need explicit freshness and refresh behavior.

Verify: Compare request count and cold/warm timings; test timeout, rate-limit, disconnect/account switch and region changes. Preserve URL allowlists, response bounds, credential isolation and safe error messages.

R29 Cancel resource work and reject late results consistently

Planned improvement | Applies to items 5, 15, 18, 44, 45, 46

Requirement: Give each app-owned task an identity, account/revision context and native cancellation signal. Check cancellation after awaits and before expensive decode or commit. Define whether an already-running transaction finishes; keep completed work durable. Closing Settings should detach its view, not cancel the app task.

Rationale: Ignoring stale responses protects state; cancelling eligible requests also saves bandwidth/CPU and makes Stop predictable. Cancellation is cooperative and must be designed for each operation [75]. Source patterns suggest a gap, not a newly reproduced user-facing failure.

Verify: Gate replies deterministically, then Stop, change account or close/reopen Settings at each stage. No late unauthorized save; no duplicate replacement task; committed results remain and eligible requests stop.

R33 Provider feasibility: platforms, favorites, DLC and Last Played

Evidence-gated investigation | Applies to items 3, 4, 5, 6, 7, 8

Requirement: Keep platform, storefront, ownership evidence and per-device installation separate. Maintain a dated capability matrix for each adapter. Import favorites only through a verified supported source; otherwise keep manual favorites and explain the limitation. Link DLC by stable base/add-on IDs, distinguish editions, demos, soundtracks and unknown relationships, and review ambiguous links. Preserve provider timestamp/source for Last Played; missing or failed refresh means unknown/stale, never a fabricated date or removal of saved data [3, 91, 92].

Rationale: The reviewed IPlayerService interface documents ownership/recent-play methods but no favorites method. Steam SDK installation checks explicitly distinguish installation from ownership; their presence does not establish permission for a third-party universal DLC scanner. Treat cross-launcher coverage as a feasibility gate [3, 91, 92].

Verify: Test installed-but-not-owned evidence, private/missing fields, unsupported favorites, free games, DLC without its base game and overlapping editions. Manual favorites and curated dates survive refresh; What to Play remains PC-only.

Phase completion gate

For each supported adapter, verify configured/authenticated/synced status, capability limits, reviewed import identities, manual overrides and DLC/favorite fallbacks. Closing/reopening Settings during library/artwork work preserves one app-owned job, pending review and committed data; cancellation and late replies behave correctly.

PHASE 03 / ITEMS 08-11

Library Data & Play Tracking

DONE ☐

08 Improve Last Played Synchronization

Current clarification: Keep the provider/source and last successful refresh. An unavailable date is unknown; retain valid history and do not infer an exact session start from cumulative playtime. R14/R33 refine this item.

Retrieve Last Played information from supported libraries and normalize it into GameVault's internal game records. This information should later contribute to:

- Recently played games
- Backlog management
- Recommendations
- Library sorting
- Play-history statistics

09 Automatic Play-Session Detection

Current clarification: Automatic PC tracking is an evidence-gated prototype. Launcher handoff, confidence, manual correction, sleep/crash recovery and normal-user permissions must pass before treating a detected duration as confirmed. R34 refines this item.

GameVault should detect when supported games are started and stopped. When a game launches:

- Identify the game.
- Detect when the game closes.
- Automatically add the session to GameVault.
- Record the session start.
- Record the session duration.

This should minimize or eliminate the need for manual session recording.

10 Launch Games Directly From GameVault

Current clarification: Use scoped launcher/executable targets and explicit user actions; a launch request is not proof that gameplay started. Preserve manual fallback when the launcher/target is missing. R05/R34 refine this item.

Allow installed games to be launched directly through GameVault. GameVault should:

- Determine the appropriate launcher.
- Launch the game through Steam, GOG, Epic, or another associated platform.
- Integrate launching with automatic session detection.

11 Intelligent Use of Play History

Combine external Last Played information with locally detected GameVault sessions. Use this information to improve:

- Recently Played
- Currently Playing
- Backlog classification
- Game activity information
- Recommendations
- Library organization

R14 Explainable playtime and resilient session tracking

Planned improvement | Applies to items 8, 9, 10, 11

Requirement: Show provider total, local sessions and baseline separately. Add a correction/reconciliation preview with undo, optional session merge and timezone-aware start/end times. Before automatic process tracking, define launch handoffs, crash and sleep/resume behavior, overlaps and multi-device ownership of a session.

Rationale: A conservative baseline protects prior totals but makes downward correction unclear. Transparent provenance makes discrepancies diagnosable; offset-aware timestamps aid future synchronization [18]. Process detection must not silently convert idle or duplicate time into playtime.

Verify: Exercise delayed provider totals, corrected baselines, overlap, sleep and a duplicate-device session without counting the same play twice.

R34 Safe game launching and automatic session detection

Evidence-gated investigation | Applies to items 9, 10, 11

Requirement: Use reviewed launcher protocols or an explicitly chosen local executable with scoped permissions and structured arguments; never build shell commands from game names. Validate install/launcher availability and explain failures. Prototype normal-user Windows start/stop events with path, process ID and creation-time evidence; follow launch handoffs to the actual game, not the launcher. Make tracking optional, show confidence, and retain manual correction. Checkpoint unfinished sessions and define sleep, crash, clock-change, overlap and app-exit behavior [93, 94, 95, 96].

Rationale: Windows provides process events, but process IDs can be reused and a launcher may spawn another process. Events establish a possible mechanism, not accurate playtime or foreground activity. Avoid invasive process injection or automatic elevation; unavailable detection should fall back visibly [93, 94, 95].

Verify: Launch from GameVault and outside it; exercise delayed handoff, duplicate executable names, crash, permission denial, sleep and restart. Record one correct session, never count launcher/idle time as confirmed play, and retain correction/undo.

Phase completion gate

Verify provenance-aware Last Played and playtime reconciliation, safe supported launch routes and optional reliable detection. External launches, handoffs, crash/sleep/restart, overlaps and corrected sessions do not double-count play. Unavailable detection remains visible with manual fallback.

PHASE 04 / ITEMS 12-14

Collection Experience

DONE ☐

12 Simplify Collection Categories

Current clarification: The four categories remain the owner's requirement. Saved filters do not add competing top-level collection categories. R05/R35 refine the interaction.

The **Your Collection** section should contain only:

1. All Games
2. Favorites
3. Completed
4. Backlog

Remove:

- Missing Covers
- Other unnecessary collection categories

Missing artwork should instead be handled through the dedicated cover-management functionality.

13 Installed & Uninstalled Separation

Current clarification: Installed status is per device and based on validated scan evidence, separate from ownership. A failed scan preserves prior evidence instead of reclassifying the whole library. R33/R35 define the exact two-section sort behavior.

Within every collection category, show two clearly distinguishable sections: **Installed** first, followed by **Uninstalled**. Both sections belong to the currently selected category. Apply the user's selected sorting method independently within each section. For alphabetical sorting, show **Installed: A → Z**, followed by **Uninstalled: A → Z**, rather than combining all games into one alphabetical list.

14 Steam-Inspired Game Side Panel

Selecting a game should open a detailed side panel inspired by Steam's library interface while maintaining GameVault's own visual identity. The panel can eventually become the central interface for:

- Game information
- Play statistics
- Achievements
- Additional metadata
- Play controls
- Sessions
- Store information
- Installation information
- DLC
- Pricing

39 Dedicated Feedback Button

Phase 4: collection/app interface. Finalize the submission workflow before launch.

Add a clearly visible Feedback button for ideas, suggestions, and general product feedback. When this work is reached, decide the destination, form fields, privacy handling, and follow-up process together with the owner. Keep the workflow distinct from technical bug reports. Selecting a paid feedback service requires the owner's decision first.

42 Dedicated Bug Report Button

Phase 4: app interface. Agree the reporting workflow together before launch.

Add a separate Bug Report button for technical problems. When development reaches this feature, choose the destination and process together with the owner. The proposed form should capture steps to reproduce, expected behavior, actual behavior, and the GameVault version; optional screenshots or diagnostics should be reviewable before sending. Redact credentials and sensitive information. Decide submission confirmation and follow-up behavior together. Any paid reporting solution requires the owner's decision first.

R05 Saved filters, bulk edits and safe custom launch targets

Planned improvement | Applies to items 10, 12, 13, 14

Requirement: Save named combinations of filters and add previewable bulk changes with undo. Evaluate a custom launch target for unsupported/manual games, explicitly reviewed by the user. Add short-session, co-op or controller filters only when their metadata is dependable. Keep the agreed collection categories.

Rationale: Saved presets reduce repetitive filtering [5]; bulk previews prevent widespread accidental edits. A reviewed launch target extends manual coverage without assuming every launcher has a reliable launch integration.

Verify: Reopen a saved multi-filter preset after restart and backup restore; undo a bulk edit and test a missing launch target.

R10 Feedback, Bug Report and reviewable diagnostics

Planned improvement | Applies to items 39, 42

Requirement: Create distinct Feedback and Bug Report actions. Bug Report should collect version, steps, expected/actual behavior and optional attachments; Feedback should accept ideas. Preview and remove logs/screenshots, redact secrets and personal data, and offer export-only diagnostics. Choose the submission destination together; show a report ID only after successful submission.

Rationale: Structured reports are easier to reproduce, and preview prevents accidental disclosure. OWASP advises excluding sensitive data from logs [14]. No external service or payment commitment is selected.

Verify: Create an offline report, remove an attachment, inspect redaction and simulate failed submission without losing the draft.

R23 Render a bounded library and reuse search work

Evidence-gated investigation | Applies to items 1, 12, 13, 14, 38

Requirement: Compare accessible pagination with windowed rendering of visible rows. Preserve search focus, keyboard position and scroll restoration. Cache searchable text and filter/sort results by relevant data revisions; schedule rapid typing so obsolete work is skipped. Start with the smallest measured change.

Rationale: The source shows repeated work that grows with matching games. Bounding rendered elements is a credible way to reduce layout and memory; CSS content-visibility alone keeps the DOM and filtering work [17]. Actual gains need WebView2 measurements [72].

Verify: Compare search latency, DOM count and scroll frames on the same fixtures; verify keyboard/screen-reader access to every result and identical sorting.

R24 Update progress without rebuilding recommendations

Evidence-gated investigation | Applies to items 14, 24, 25, 26, 38, 45 | Sources [27, 60, 61]

Requirement: Update progress text, counters and Stop state directly. Rebuild recommendations only when their input data or discovery seed changes; reuse a profile/index keyed to those inputs. Rebuild visible achievement rows only when their data, filter, page or sort changes.

Rationale: Progress updates need not repeatedly rescore the collection or replace unchanged controls. This may reduce background CPU and focus disruption, but timings must establish its value; caching needs explicit invalidation.

Verify: Record background CPU and refresh work before/after. Edit sessions, ratings and genres; switch accounts, filters and languages; confirm fresh results and stable focus.

R32 Reuse suggestions for local GameVault searches

Evidence-gated investigation | Applies to items 12, 14, 18, 27 | Sources [60, 61, 78]

Requirement: Reuse one accessible popup component with separate data adapters for owned games, saved Wishlist, session/timer game pickers, achievements and existing tags. Use saved IDs and local indexes offline. Consider a grouped global search later only if the owner wants it.

Rationale: Local suggestions are feasible without a hosted search service. A shared control can improve consistency, while each surface keeps its own filters and actions. Do not introduce a dropdown where a short existing list works well.

Verify: Compare finding a game with current filtering; test edits/imports/removals, duplicate names, empty collections, large libraries and offline use. Preserve each surface's scope, focus and explicit action.

R35 Collection layout, categories and Settings interaction

Planned improvement | Applies to items 1, 12, 13, 14, 44

Requirement: Retain All Games, Favorites, Completed and Backlog as the collection categories; saved filter presets are controls within them. Put Installed before Uninstalled and apply the selected sort independently to each section, with deterministic ties and a clearly explained unknown scan state. Keep per-device installations from duplicating a canonical game. On wide screens use a bounded detail side panel; on narrow windows use an accessible full-width detail view. If Settings is modal, contain focus, label it, support Escape/Close and return focus; a non-modal panel must not claim modal semantics [37, 104].

Rationale: Responsive panels must leave search and actions reachable at high DPI and enlarged text. APG specifies modal focus behavior; it does not require every Settings design to be modal. A failed scan should not falsely mark every game uninstalled [37, 104].

Verify: Verify installed A-Z then uninstalled A-Z in every category, stable selection after refresh and no duplicate game rows. Test keyboard-only navigation, focus return, screen readers, small/ultrawide windows and unsaved settings.

Phase completion gate

Verify exactly four categories with independent Installed/Uninstalled sorting, detail panels, saved filters, undoable bulk edits and scoped local suggestion adapters. Feedback and Bug Report destinations are agreed with the owner; offline drafts, preview/redaction and failed submission preserve user data.

PHASE 05 / ITEMS 15-17

Cover & Artwork System

DONE ☐

Current cover direction: SteamGridDB is the only automatic game-cover provider; custom uploads remain supported. The earlier AI/store-cover wording below is retained from the original roadmap as historical planning text and is superseded by this direction.

15 Improve "Find Missing Covers"

Current clarification: Current direction supersedes the preserved AI-cover sentence: SteamGridDB is the sole automatic game-cover provider, with custom uploads. Report no missing work as success/no-op; distinguish unmatched, unavailable and failed results. R04/R27 apply.

Move **Find Missing Covers** next to the **Grid / List** controls above the game library. When a search completes, clearly report: **Successfully fetched covers** and **Covers that could not be found**. The process should not report a failure simply because there were no missing covers to retrieve. This should also work correctly when AI-generated covers are enabled.

16 Improve Artwork Source Handling

Current clarification: Current direction supersedes the original store-cover alternative: retain SteamGridDB plus custom uploads and attribution. A future source requires an explicit owner decision and rights/access review; removed providers must not return. R04/R27 apply.

SteamGridDB can remain a primary configurable artwork source. GameVault should also use appropriate official/store artwork where relevant. Artwork handling should be standardized so that future sources can be added without rewriting the library UI.

17 Discover New Games Artwork

Current clarification: Provide recognizable discovery thumbnails through the agreed artwork route or a permitted, explicitly approved future source. The original store-listing idea is historical, not authorization to restore automatic Steam/store covers. R04/R27 apply.

Games displayed under **Discover New Games** should include small cover images. Where practical, these should be retrieved from the game's official store listing.

R27 Separate artwork files from collection metadata

Evidence-gated investigation | Applies to items 15, 16, 17, 38 | Sources [60, 61, 62, 63]

Requirement: Evaluate a local asset cache addressed by stable hashes, with lightweight record references, size/retention limits and safe orphan cleanup. Keep original attribution. Design atomic migration, missing-image fallback and portable JSON export/import so backups still include required artwork.

Rationale: Base64 adds about one third to binary size before other overhead. Separating bytes could reduce JSON copying and transfer and postpone the cap; decoded-image memory is separate. Extra files introduce recovery/backup complexity, so this is conditional research.

Verify: Measure metadata bytes, asset bytes and total process memory separately. Restore offline on another installation; test interrupted migration, missing/corrupt files, duplicate images and rollback.

Phase completion gate

Verify the SteamGridDB/custom-upload direction, credits, no-op and separate missing/unavailable/failure counts. Discovery thumbnails follow approved source rights. Artwork replacement and any measured asset-cache migration preserve offline recovery and portable backups; no removed provider returns.

PHASE 06 / ITEMS 18-20

Achievements

DONE ☐**18 Build the Achievement Data System**

Retrieve achievement information from supported platforms and convert it into a standardized GameVault format. The internal system should account for:

- Achievement names
- Descriptions
- Unlock date/time where available
- Achievement icons
- Unlock state
- Global completion statistics where available

19 Add Achievements to Game Pages

Each supported game should have an Achievement section displaying:

- Achievement artwork
- Achievement description
- Locked achievements
- Overall completion percentage
- Achievement name
- Completed achievements
- Unlock information

Locked achievements should appear visually greyed out.

20 Global Achievement Comparison

Current clarification: Global percentages describe unlock rarity, not player skill. Unavailable rarity must not erase personal progress or masquerade as 0%. R17/R36 refine this item.

Where the source platform provides the information:

- Display global unlock percentages.
- Compare the user's achievement completion against global completion statistics.

R17 Achievement coverage and refresh transparency

Planned improvement | Applies to items 18, 19, 20

Requirement: Show each game's last successful refresh, account/source and specific unavailable/private/unsupported reason. Make progress and rarity refresh status distinct. Explain limited automatic/manual batches and let users resume remaining work. Add platform coverage only through legitimate APIs; keep manual entries separate from provider progress.

Rationale: Saved progress can look current when a provider is unavailable, and missing achievements can be mistaken for zero progress. Existing caching should remain; clearer status makes it trustworthy. Non-Steam access needs service-specific feasibility checks [32].

Verify: Simulate a private profile, failed rarity request, account switch and partial batch; preserve progress and hidden-achievement protection.

R36 Achievement identity, private states and completion denominators

Planned improvement | Applies to items 18, 19, 20

Requirement: Join game schema, player unlock state and global percentages by provider + app ID + achievement API name. Store progress and rarity refresh times separately. A failed/private request is unavailable, not zero unlocked. Define completion over the valid schema: no achievements is not a misleading 0%; a missing schema has an unknown denominator. Preserve hidden-achievement spoilers, localized labels and manual non-Steam entries, and do not upload or claim to unlock achievements through GameVault [97].

Rationale: Valve exposes schema, user achievements and global percentages as distinct methods. Their results can differ in availability and freshness; an API name is safer than a translated display label for joining them. Global rarity is context, not a personal skill ranking [97].

Verify: Test zero-achievement games, missing schema, private profiles, unavailable rarity, changed language and account switching. Cached progress survives failure; denominators, hidden content and manual/provider separation remain correct.

Phase completion gate

Verify provider/app/achievement identity, schema denominators, locked/hidden states, localization, progress/rarity freshness, account isolation and separate manual entries. Failed/private/partial batches preserve valid cached achievements and support resuming remaining work.

PHASE 07 / ITEMS 21-23

Pricing & Library Value

DONE ☐

21 Build the Pricing System

Create a system capable of retrieving pricing information for games. The system should:

- Retrieve current prices.
- Associate pricing with individual stores.
- Identify which platform/store each owned copy belongs to.
- Establish a data model capable of supporting historical pricing.

22 Track Purchase Information

Current clarification: No universal receipt/purchase-price API is verified. Keep manual entry, unknown amounts and source-specific feasibility in scope; current sale price must not substitute for paid cost. R15/R37 apply.

Where the information can reliably be obtained, record:

- Purchase location
- Purchase price
- Purchase date

When this information cannot be retrieved automatically, GameVault should eventually allow users to manually enter or correct it.

23 Calculate Library / Account Value

Current clarification: Show original currencies, dated conversion estimates where supported, coverage and missing values. MSRP/replacement value is an estimate; recorded spend requires actual cost evidence. R15/R37 apply.

Use the pricing system to calculate useful library statistics. Potential values include:

- Current library value
- Total amount actually spent
- Original/MSRP value
- Current replacement value
- Total savings
- Average price paid per game

Only values supported by reliable data should be presented as definitive.

R15 Price provenance, editions and reliable comparisons

Evidence-gated investigation | Applies to items 21, 22, 23, 27, 28

Requirement: Show offer age, requested region versus returned currency, edition/DLC relationship and unavailable-price reasons. Before price history or cross-store value totals, model amounts/currencies per record; separate paid cost, current offer and estimated replacement value. Convert only with a dated exchange-rate source, or compare within one currency.

Rationale: A relabeled amount is not a conversion, and stale or edition-mismatched prices mislead comparisons. Existing public store formats can change. Evaluate provider coverage/permissions first; IsThereAnyDeal restricts competing apps and may require approval [19]. It is a candidate, not an approved dependency.

Verify: Test currency fallback, unavailable/stale prices and different editions; verify that imported offers never overwrite purchase costs.

R37 Reliable purchase records and honest library valuations

Planned improvement | Applies to items 21, 22, 23

Requirement: Store monetary amount, original currency, product/edition, store, date and source per record using integer minor units or exact decimal rules. Keep unknown cost separate from a known free purchase. Support manual receipts/corrections, bundles with explicit allocation, gifts, refunds and duplicate owned copies. Display recorded spending separately from current replacement estimates and MSRP with dated sources. Show valuation coverage and missing records. Compare within currency or use a dated, named rate with rounding; changing a label must never convert money [19, 98, 110].

Rationale: Owned-game lists and current offers do not prove what a user paid. ECB rates are reference information, not transaction quotes; using them for a display estimate must be labelled. Provider terms/access need approval checks before integration; no price API has been selected here [98, 19].

Verify: Test mixed currencies, zero/unknown amounts, delisted products, stale offers, editions, gifts, bundles and refunds. Demonstrate totals with known coverage and verify sale checks never overwrite purchase costs.

Phase completion gate

Verify exact store/edition/regional offer context, monetary currencies and reliable paid-cost entry/correction. Valuations show coverage, unknown costs and dated estimates; bundles/refunds and duplicate copies are explicit. No sale lookup overwrites recorded spending or merely relabels currencies.

PHASE 08 / ITEMS 24-26

Recommendations & Discovery

DONE ☐

24 Rework "What to Play"

Current clarification: Preserve the existing explanation/exploration foundations. Evaluate relevance, useful surprise, coverage and repetition together; increasing randomness is not the completion criterion. R16/R38 apply.

Improve the recommendation algorithm so that it does not simply favor the most obvious games. The system should:

- Increase randomness.
- Surface backlog titles.
- Use play history intelligently.
- Occasionally surface forgotten games.
- Avoid repeatedly recommending the same games.
- Introduce more unexpected selections.

The goal is to make **What to Play** useful when the user does not know what they want to play.

25 Recommend Games the User Doesn't Own

Increase the frequency with which GameVault recommends games outside the user's existing collection. These recommendations can eventually use information such as:

- Existing library
- Recently played games
- Wishlist
- Favorites
- Genres
- Achievement activity
- Playtime
- Similar games

26 Clean Up "Discover New Games"

Remove the duplicate **Refresh Game Suggestions** control at the bottom of the section. Keep only the refresh control in the **top-right corner**. Integrate the improved artwork system so recommended games have recognizable cover art.

R16 Useful alerts and controllable recommendations

Planned improvement | Applies to items 24, 25, 26, 28, 29

Requirement: Add absolute target prices, per-game mute, snooze/quiet hours and channel preferences. Design closed-app/email alerts separately, with infrastructure and cost reviewed first. Add Hide this game, Less like this, reset and a configurable recent-pick history. Keep security alerts separate from optional sale messages.

Rationale: Build on existing explanations and deduplication rather than re-add them. Better controls reduce unwanted alerts and repeated suggestions. Region/currency/product context remains essential [19]; longer recommendation history should stay local unless the user chooses sync.

Verify: Mute/snooze an alert, preserve its rule after restart, and hide a recommendation without changing ownership or playtime.

R38 Evaluate recommendation usefulness, variety and repeat control

Planned improvement | Applies to items 24, 25, 26

Requirement: Keep PC-only What to Play and separate owned-game choices from unowned discovery. Add a configurable recent-pick history, Hide/Less like this/reset controls and an explainable cold-start fallback. Balance relevance with backlog coverage, genre/franchise variety and forgotten titles. Validate metadata before adding co-op/controller/short-session filters. Use one top-right discovery refresh. With deterministic fixtures, compare eligible-catalog coverage, repeats and diversity alongside task completion and owner/user usefulness feedback [99].

Rationale: The primary evaluation library distinguishes relevance, coverage, novelty and diversity. None establishes an optimal GameVault exploration percentage. Randomness alone can surface irrelevant choices; offline metrics cannot substitute for usability feedback. No paid model or hosted recommender is required by this proposal [99].

Verify: Test empty/small libraries, no-history users, hidden games, missing metadata and seeded repeated draws. No impossible pick or hidden-game leak; compare usefulness and variety without covert telemetry or changing ownership.

Phase completion gate

Verify PC-only owned-game eligibility and clearly separate unowned discovery. Hide/Less like this/reset, recent-pick history and one discovery refresh work. Compare seeded coverage/repetition/diversity with task usefulness and cold-start behavior without requiring paid recommendation services.

PHASE 09 / ITEMS 27-29

Wishlist & Online Notifications

DONE ☐

27 Complete the Cross-Store Wishlist System

Allow users to wishlist a game independently of one particular storefront. GameVault should then track which stores/platforms the user is interested in purchasing it from.

28 Wishlist Price Monitoring

Monitor prices across supported stores. Allow users to define notification thresholds. For example: **Notify me when this game is discounted by 20% or more**. Discount thresholds should support the previously planned range of approximately: **5% → 100%**

29 Online & Email Notifications

Current clarification: Closed-app/email/mobile delivery needs separate approved infrastructure and consent. Do not promise that a desktop task runs after the whole app exits or that OS delivery is immediate. R39 applies.

Once the online infrastructure is available, notify users when their wishlist conditions are satisfied. This could eventually support:

- Email notifications
- Desktop notifications
- Mobile notifications
- GameVault in-app notifications

R31 Wishlist search suggestions while typing

Evidence-gated investigation | Applies to items 27, 28

Requirement: Add a manually selected suggestion dropdown: typing Dia can show matching Diablo titles returned by the selected store. Use a lightweight, delayed title/ID lookup; resolve the exact product and current offer only after selection. Keep Search stores and the existing wishlist review/save step.

Rationale: The W3C editable-combobox pattern fits this interaction [80, 81]. Reusing the full price-enriched search on each keystroke would add unnecessary work. The detailed study below covers safe reuse and provider limits.

Verify: Verify rapid typing, outdated replies, store/region switches, offline/error states, keyboard/screen-reader use and exact product identity. Suggestions must never auto-add or buy a game.

R39 Closed-app alerts, delivery preferences and bounded monitoring

Evidence-gated investigation | Applies to items 2, 28, 29, 40

Requirement: Preserve thresholds in 5% steps and add optional absolute target prices. Match exact product, edition, store, region and currency; a 100% threshold needs a verified free offer, not an absent price. For closed-app monitoring, design an owner-approved always-available service or clearly bounded OS task, with persistent rules, deduplicated delivery IDs, expiry/recheck of offers, retry limits and Retry-After handling. Offer per-game/channel mute, snooze, quiet hours, email verification/unsubscribe and contextual mobile permission; denied permission leaves in-app history usable. Keep security messages separate [101, 102, 106].

Rationale: Android persistent scheduling supports eventual background work but is not a general immediate-execution promise. Notification permission can be denied. Hosting/email/SMS may cost money and require the owner before activation; delivery timing and coverage must be stated honestly [101, 102].

Verify: Exercise a closed client, delay/retry, withdrawn deal, duplicate event, timezone/DST quiet hours, permission denial and unsubscribe. Produce no unwanted duplicate or stale-price alert; verify rules survive restart and data restoration.

Phase completion gate

Verify manual-selection Wishlist suggestions, exact products/duplicates, accessible keyboard/IME behavior, stale replies and full-search/offline fallback. Verify threshold/target-price rules, consent/mute/quiet hours and deduplicated contextual delivery. Closed-app/hosted work needs approved access/cost and proven lifecycle limits.

First-Time Setup Experience

DONE ☐

30 Build the First-Time Setup Wizard

Current clarification: This revision supersedes mandatory Connect at least one library with a local/manual skip route. Keep connection, account and SteamGridDB setup optional; preserve the original connected-library flow for users choosing it. R40 applies.

When GameVault is installed for the first time, guide the user through a structured onboarding process. Recommended flow: **Welcome to GameVault** → **Choose Game Libraries** → **Connect At Least One Library** → **Enter Required API Keys / Authentication** → **Choose Which Libraries Should Be Imported** → **Configure SteamGridDB** → **Initial Library Sync** → **Initial Cover Fetch** → **GameVault Ready**

This setup should only be required for a fresh installation. Normal GameVault updates should not force users through the initial setup process again.

31 Secure Credential Handling

Current clarification: Retain Windows protected storage and excluded backup keys. Reconnection is expected after device-bound restore; future native-browser OAuth uses PKCE and protected per-platform storage. R08 applies.

API credentials and authentication information should be handled appropriately and securely. GameVault should clearly communicate:

- What information is stored.
- Where it is stored.
- Whether information remains local.
- Which credentials users should retain themselves.

Users should be informed that they are responsible for retaining copies of API keys or other credentials that may be required again after reinstalling GameVault or if a future migration requires them.

41 VirusTotal Launch Scan & Release Evidence

Release gate: after the final launch build is packaged and before public distribution.

Run the final distributable installer and relevant executables through VirusTotal when ready for launch, subject to applicable service terms and the payment rule. Record the version, scan date, SHA-256 file hash, report link, and outcome. Investigate detections and document any vendor-confirmed false positives. Re-scan changed release files. Describe the result as a VirusTotal scan report, rather than a security certification or guarantee. VirusTotal aggregates vendor results and does not issue its own verdict. Public submissions may be shared: use only release artifacts intended for distribution, without secrets or private user data. References: VirusTotal How it works; False positive guidance; Public vs Premium API.

R08 Extend protected credentials to future sign-in

Evidence-gated investigation | Applies to items 4, 31, 36, 37

Requirement: Retain this protection. For future OAuth integrations, use the system browser, authorization code and PKCE; keep account tokens separate from store keys. Add expiry/revocation handling and secret-redaction checks. Document reconnection after restore and choose native protected storage for each future mobile platform.

Rationale: Protected Windows key storage is already implemented; the gap is the lifecycle and additional platforms. RFC 8252 supports browser-based OAuth and PKCE [11]. Device-bound protection is not a credential-transfer solution [12].

Verify: Expire/revoke a token and restore on another device; inspect exports, logs and diagnostics for secrets.

R09 Publisher signing and update recovery

Evidence-gated investigation | Applies to items 32, 41

Requirement: Preserve the current updater gate. Evaluate Windows publisher signing and key rotation; document recovery after an interrupted install or incompatible migration. Test restoration before promising rollback: older app versions may not understand newer data. Keep version, signature and SHA-256 evidence tied to exact release files.

Rationale: Updater integrity and Windows publisher identity solve different problems. Microsoft explains reputation/signing behavior; certificates do not guarantee warning-free installs [13]. Any signing cost requires the owner's decision.

Verify: Reject altered packages, recover from an interrupted update and prove a supported data-restoration path across versions.

R11 Extend the existing release gate and maintenance record

Planned improvement | Applies to items 38, 41, 43, 46

Requirement: Add periodic dependency/code review with a short record of fixes and measurements. Prioritize remaining hands-on cases: real accounts, native dialogs, clean normal-user installation and wider supported Windows versions. Before launch, scan the exact release artifact on VirusTotal, retain its hash/date/report, investigate detections and rescan changed packages.

Rationale: Extend an established gate rather than duplicate it. SSDF supports ongoing maintenance [15]. VirusTotal shares submitted files and aggregates vendor verdicts; a clean report is evidence, not certification [20, 21]. Do not automate commercial use through its restricted public API [22].

Verify: Link each release to its tested source and artifact hash, retain known issues, and close or explicitly accept outstanding hands-on failures.

R18 One source for release and setup guidance

Planned improvement | Applies to items 30, 31, 38, 41 | Sources [23, 24, 33]

Requirement: Drive current-version links and installer names from one release record, or use stable Latest links. Add a release check for version drift across setup, status, usage and feature guides. Label older reports as historical and preserve their exact test totals. Include a clear local/offline introduction before optional provider setup.

Rationale: A correct build can still send users to an older installer. Consistent guidance reduces support effort and helps users understand API keys, preview versus desktop storage, artwork choices and alert limits. This is a verified documentation discrepancy, not a runtime failure.

Verify: Follow every current setup/download/update link from a clean browser and confirm all refer to the verified release or explicitly labeled history.

R40 Resumable first-run setup without forced accounts or libraries

Planned improvement | Applies to items 30, 31, 36

Requirement: Offer Welcome -> local/manual or connect libraries -> verify chosen connection -> choose imports -> optional SteamGridDB setup -> app-owned sync/review/artwork -> Ready. Save completed choices and resume unfinished steps without duplicate imports. Let users skip or revisit optional steps from Settings. Make missing/invalid keys and offline state repairable; do not silently claim a provider is connected. Detect existing collection/setup state so updates and restores do not restart onboarding. Explain desktop versus browser-preview storage and reconnection after reinstall [103, 3, 11].

Rationale: Apple recommends short, skippable, contextual learning; its game-specific guidance is applied here as a design inference for a library utility. Requiring a connected library or account would unnecessarily block the already supported local collection [103].

Verify: Test a fresh local-only install, one failed provider, canceled artwork setup, restart midway, normal update and restored collection. Preserve choices/data; only a deliberately requested reset repeats initial setup.

Phase completion gate

Verify resumable optional setup, existing-install/restore detection, protected credentials, account/factor recovery and revoked remembered devices. Final desktop release gates cover installer, signatures, exact artifact hashes, scan-report limits, recovery, guide-version consistency and real-account/native coverage; owner controls all costs.

PHASE 11 / ITEMS 32-35

Mobile GameVault

DONE ☐

32 Stabilize the PC Version First

The desktop application should reach a sufficiently stable architecture before full mobile development begins. This reduces the risk of simultaneously maintaining two applications while GameVault's underlying systems are still changing significantly.

33 Build the Mobile Data & API Layer

Establish the infrastructure necessary for the mobile application to communicate with the wider GameVault ecosystem. Potential synchronized information includes:

- Library
- Sessions
- Account information
- Favorites
- Achievements
- Cross-platform ownership
- Wishlist
- Game metadata

34 Build the Mobile Application

Current clarification: Phone/tablet navigation adapts to available space; PC launcher paths/process detection do not become universal mobile features. Prove target-platform/plugin capability before implementation. R41 applies.

Create a dedicated mobile GameVault experience. The mobile interface should be designed specifically for phones and tablets rather than simply shrinking the desktop interface. Mobile should have its own appropriate navigation and platform filtering.

35 Cross-Device Synchronization

Current clarification: Sync records/revisions and durable edits, not a live SQLite file. Define conflicts, deletion, replay and duplicate session handling; timestamps alone do not settle competing edits. R03/R41 apply.

Once both applications are available, establish synchronization between: **GameVault PC** ↔ **GameVault Mobile** Changes to supported information should propagate appropriately between devices. This could include:

- Favorites
- Sessions
- Wishlist
- Collection information
- Game metadata
- User preferences

R03 Offline safety before cross-device sync

Planned improvement | Applies to items 2, 33, 35

Requirement: Retain offline use without requiring an account. For future sync, add stable record identifiers and revisions, pending-edit visibility, conflict review and deliberate deletion propagation. Synchronize validated records or snapshots rather than a live SQLite file.

Rationale: Independent offline edits can overwrite notes, duplicate sessions or resurrect deleted games. Timestamp formatting [18] helps exchange records but does not decide whose edit wins.

Verify: Reconnect two devices after conflicting edits, a deletion and interrupted sync; resolve them without losing unrelated records.

R41 Mobile contracts, conflict-safe sync and device-specific capabilities

Evidence-gated investigation | Applies to items 2, 3, 32, 33, 34, 35

Requirement: Define a versioned, authenticated HTTPS record API with stable IDs/revisions, record-level authorization, validated bounded payloads, conflict/precondition handling and idempotent mutation IDs. Sync committed records and asset references with a durable offline outbox; use deletion markers and explicit resolution for competing notes/sessions rather than relying on device clock order. Keep install paths, active processes and local provider credentials device-scoped. Model PC/Mobile/All/Other separately from storefront ownership. Build adaptive touch navigation for current window size, with native protected storage and OS-compliant background work; prove the chosen framework/plugins on each target before committing [100, 105, 108, 109, 111].

Rationale: Offline guidance requires local/network reconciliation. Android adapts to changing window sizes and restricts visibility of other apps. These sources support capability-specific design, not a universal mobile scan or automatic PC launching from a phone. Safe API failures can use structured problem types without exposing secrets [107].

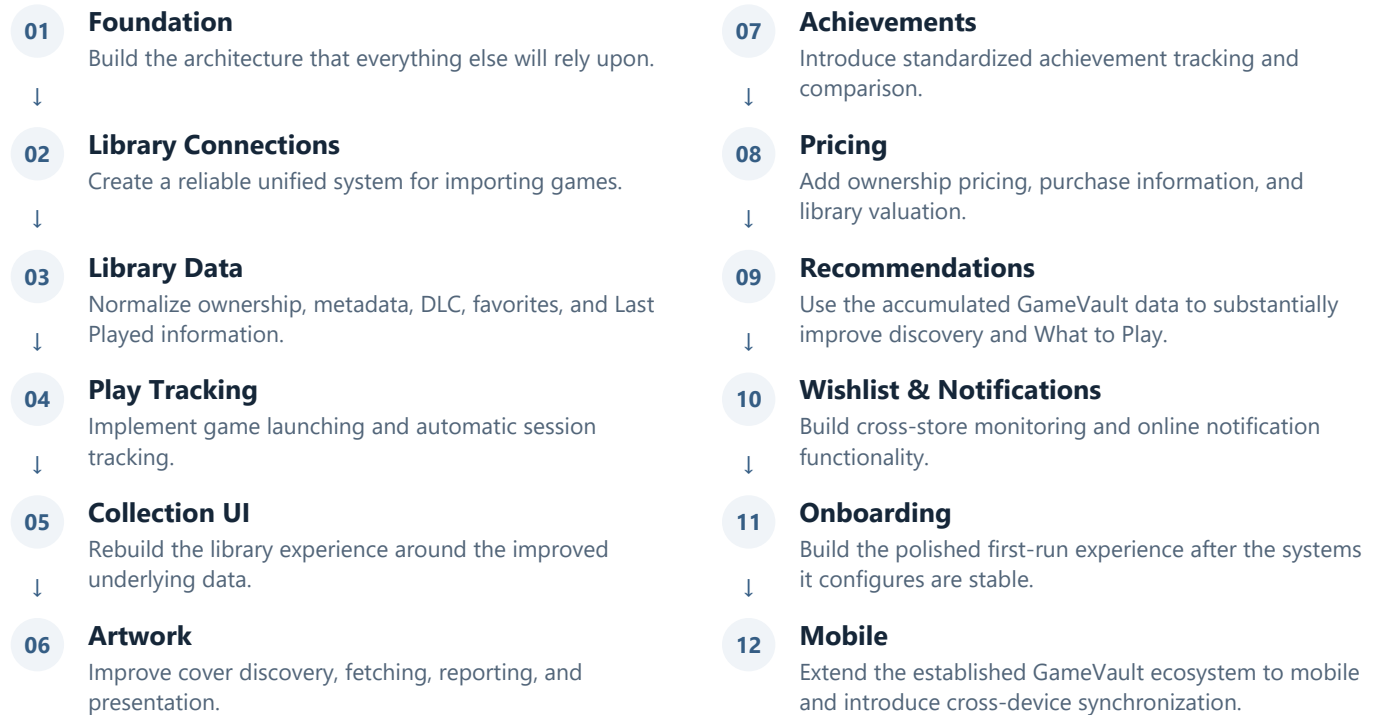
Verify: Sync two offline devices after conflicting edits, deletion and duplicated session upload; replay a mutation, deny another account access and interrupt asset transfer. Test phone/tablet/rotation, revoked login and older-client schema compatibility without losing unrelated records.

Phase completion gate

PC stability gates pass before mobile expansion. Verify versioned authorized APIs, local-first read/write queues, conflict/deletion/replay/session deduplication and device-specific fields. Phone/tablet/adaptive UI, protected tokens, permission denial, offline restore and older-client compatibility pass on the chosen supported targets.

Recommended development sequence

The overall development path should therefore be:



06 → 07

Development priority

The immediate development focus should be:

Phases 1-3

Architecture & Platform Foundation

Library Connection System

Library Data & Play Tracking

These systems form the foundation for almost every major feature planned afterward.

Achievements depend on reliable platform connections.

Pricing depends on reliable ownership and store information.

Recommendations become substantially better once GameVault understands play history.

Wishlist notifications require online infrastructure.

Mobile requires a stable shared data model.

The first-time setup wizard depends on the final library connection and authentication workflows.

Prioritizing GameVault's underlying architecture before expanding the visible feature set should make the application cleaner, more reliable, and easier to maintain.

Expanded delivery order: Foundation/data/recovery and job ownership come first; validate provider access before feature promises. Establish the performance baseline before choosing R23-R30, then preserve the original dependency path. Account/API designs start early, while optional onboarding and mobile wait for stable workflows. Maintenance, accessibility, privacy, costs and regressions apply throughout.

Performance delivery and benchmark plan

Historical source inspection is retained. These optimization candidates have been promoted as measured investigations, not promised speedups or an application rewrite.

Reviewed scope: Code snapshot: 776f294f (0.3.9), reviewed 5 October 2026. Latest public release checked: 0.3.8. Findings come from source inspection; no application files, live collection or installed app were changed, and no performance benchmark or new app test was run.

Existing architecture: Stack: a vanilla JavaScript/HTML/CSS frontend, Tauri 2 with Rust, SQLite through rusqlite, reqwest HTTP and Windows WebView2. Optimize measured paths inside this architecture before considering a rewrite [60, 63, 65, 68].

Preserve: Keep the existing playtime indexes, lazy image loading, narrow timer/translation updates, achievement pagination, four-at-a-time icon fetches, compressed images, bounded scans/responses, credential protection, transactional conflict checks and previous-save recovery. The 0.3.9 reload/unfinished-edit fixes are existing work from another development chat, not proposed or implemented here [60, 61, 63, 64, 65].

Investigation order: Suggested investigation order: establish R13 measurements; try bounded library rendering (R23), narrow progress updates (R24), native database threading (R26) and HTTP reuse (R28). Then assess repeated saves (R25), artwork separation (R27), cancellation (R29) and gradual cleanup (R30). Reorder using measured results and owner choices.

Comparison method: Use deterministic exported/generated fixtures in isolated profiles and databases. Record exact build/commit, Windows/WebView2, hardware, power mode, window/DPI and file bytes. Compare cold and warm runs separately; run enough repetitions to report median, a meaningful tail percentile and sample count. Keep raw traces and failures. Use Performance marks plus browser traces [72, 73] and native timing around commit boundaries; browser mocks do not measure native HTTP or SQLite.

Measure	Scenario and evidence
Startup and search	Ready-to-use startup and keystroke-to-painted-results; repeated queries/sorts on realistic, 1k, 10k and 50k metadata fixtures. Separate filtering/sorting/DOM/layout cost; large stress fixtures are not support claims.
Scroll and accessibility	Rendered DOM count, frame timing and focus restoration in grid/list/achievement views. Compare pagination/windowing with current behavior; every result remains reachable.
Save and recovery	Time clone, validation, stringify, IPC, native parsing, connection setup and transaction commit at small and near-25-MiB sizes. Test lock waits, concurrent edits, crashes and failed writes.
Artwork and memory	Metadata-only versus artwork-heavy fixtures; JSON/cache bytes, decode peaks and combined native/WebView2 process memory. Check growth after repeated navigation/sync and offline restore.
Background/provider work	Progress-only redraws, request count, connection reuse and CPU during gameplay/minimized/idle states. Delayed/offline/rate-limited fixture responses; account/region changes and Stop at each boundary.
Decision and regression	Set latency/memory/CPU budgets after baseline collection. Compare the same fixtures/build conditions; retain correctness and recovery gates. Accept changes only with useful measured gains and no material behavior regression.

Limits: No percentage speedup, vulnerability finding or hard performance target is claimed. HTTP reuse and main-thread command behavior have documented mechanisms; their actual impact is unmeasured. Larger schema/cache migrations add risk and require separate design. Code cleanup is a maintenance benefit, not a speed guarantee.

Wishlist suggestions: delivery specification

Recommendation: an editable search field with a suggestion dropdown and manual selection. Typing Dia may show Diablo-related titles available in the selected store/region, plus other matching titles. This is an illustrative query, not a promise of specific catalog results. Keep the typed query until an explicit choice; never silently replace it or add a game [80].

Feasibility: high for the UI and local-data reuse; conditional for reliable online catalog coverage. Published 0.3.9 already has Steam/Epic/GOG search adapters, region selection, identity/URL checks, duplicate handling and stale-reply tokens [78, 79]. The browser preview currently has no live store search. No application code or new app tests were changed/run for this research.

Flow: Proposed flow: type Dia -> show a short title list -> choose an exact store product -> validate details/region -> open existing wishlist alert review -> explicitly Save. Enter with no selected suggestion still runs the full search. If no suggestion is available, manual/full search remains usable.

Design area	Required behavior and validation
Suggestion lookup	Prototype with at least two characters, about 300 ms after the last completed input, and 6-8 visible suggestions total. Reuse validated cached candidates promptly; refresh through a bounded native lookup. Tune these values from latency/request measurements. Empty/short input sends no live request.

Design area	Required behavior and validation
Lightweight product data	Return title, store, stable product ID, Epic namespace where needed, safe official URL and verified type when available. Steam's existing flow performs a store query plus up to 12 per-item details calls; do not repeat that price-enriched chain while typing [79]. Verify base-game/type and exact identity before addition.
Cache and stale replies	Key a bounded, short-lived cache by query, store, country and provider language; include currency when offers are cached. Invalidate pending work on edits, scope changes and closing the view. Accept replies only for the current generation/context; never reopen a dismissed popup from a late response.
Native cancellation and fallback	Cancel eligible native requests explicitly; a browser AbortController does not cancel Tauri invoke work. Browser fetch can use AbortSignal [88]. Keep token checks even when cancellation exists. Show loading, empty, offline and per-store failure states separately; do not treat an outage as no matches.
Ranking and identity	Use a case-insensitive comparison copy, preserving the displayed title. Rank exact-title/prefix and word-prefix matches before substring matches, using stable provider order as a tie-breaker. Work only with actual returned/cached titles; prefix filtering a truncated list cannot guarantee catalog completeness. Keep cross-store copies and editions distinct by IDs; add store and owned/wishlisted badges.
Selecting and confirming	A pointer click or Enter on an active option selects that product and fills its title. Fetch/validate required details before opening the existing alert dialog; no background overwrite of user fields. Recheck duplicates at Save. Prices may be unavailable; do not fabricate an offer or confuse title matching with edition ownership.

Provider access: Provider strategy: begin with the existing native adapters and fixture-based contract checks, not a new paid search service. No published, supported third-party autosuggest contract was verified here for the exact storefront endpoints already used. Their current use is evidence of an implementation, not guaranteed permission, stability or prefix completeness. Review provider terms/rate limits before increasing automatic traffic; retain manual/offline fallback. Epic's documented Ecom offering is scoped to a developer's configured product, so it is not evidence of an all-store search API [86].

Optional catalog: Optional Steam-only alternative: Valve documents IStoreService/GetAppList with a Web API key, pagination and incremental updates [84]. A local title index could support predictable prefix matching, but adds download/storage/freshness work and still needs region/type verification. The older ISteamApps/GetAppList is deprecated [85]. Keep a catalog cache separate from the 25-MiB collection; protect user keys and never embed publisher credentials. This is not needed for an initial suggestion UI and was not exercised with a real key.

Suggestion accessibility and local reuse

Interaction: Use a labelled input with combobox semantics, aria-autocomplete=list, aria-expanded and aria-controls; a listbox popup has stable option IDs and aria-selected. Keep input focus and use aria-activedescendant only for a visible active option [82]. Put extra actions outside listbox options. Arrow keys navigate; Enter chooses an active suggestion; Escape closes and preserves text; Tab continues normal focus without accepting a suggestion. Preserve ordinary cursor/editing keys [80]. Announce counts/loading/errors politely, without reading the entire list on each keystroke. Test WebView2 with assistive technology; APG examples are guidance, not production certification [81].

Control choice: A plain HTML datalist is a small prototype option, but its styling, zoom/high-contrast behavior and some screen-reader combinations have documented limitations [83]. A controlled combobox better supports GameVault themes, store/ownership badges and consistent behavior, provided accessibility is implemented and tested. Avoid automatic first-match or inline completion in the initial design.

Input/privacy: Do not search during unfinished IME composition; handle committed text after compositionend and avoid treating its Enter as selection [87]. Preserve original Unicode titles and the user's text; use locale-aware ordering [89], with an explicitly tested matching policy for accents and punctuation. Local searches stay local. Explain that live suggestions transmit the query and selected region to stores; provide a way to disable live suggestions while retaining manual search. Persist recent searches only if the owner chooses that optional feature, with clear/disable controls.

Surface	Behavior and limits
Saved Wishlist / Library	High feasibility; local/offline. Suggest matching saved games within current filters; selecting opens/focuses the exact saved item. Keep the existing live filtering and its performance work (R23).
Session / timer game picker	High feasibility; search owned games, then select a stable game ID. Selection alone never starts a timer or saves a session. Useful where a long select list becomes awkward.
Achievement game filter	High feasibility; suggest collection games and change the game filter after selection. Preserve page/sort and handle removed or renamed games; do not imply remote achievement-text search.
Existing tags / manual title entry	Moderate; offer existing tags without forcing a preset. A manual game title can suggest local matches or an explicitly requested metadata lookup; never merge games or replace notes/artwork just because a title matches.

Surface	Behavior and limits
Global search / command palette	Feasible later; separate Games, Wishlist and explicit actions, with clear scope. Ctrl/Cmd+K currently focuses the active search field [60]; any global replacement is an owner decision. No launch/delete/save on highlight.
Check	Required outcome
Typing and ordering	Dia -> Diab -> Diablo, paste/backspace/clear, two-character titles and out-of-order replies. Only the current query appears; result count/cache size/request rate stay bounded.
Scope and product safety	Switch store, All stores, region and language mid-request. Test same title/different IDs, editions/DLC, already owned/wishlisted entries and deletion before Save. Preserve exact identity and duplicate rules.
Keyboard and accessibility	Arrows, Enter, Escape, Tab and normal editing; pointer selection through blur; IME Enter; screen reader; 200% text/high DPI. No implicit choice, lost input or inaccessible result.
Network and lifecycle	Delay/offline/timeout/rate limit, malformed or partial provider replies; close/reopen Settings or navigate away while requests wait. Ignore obsolete replies; preserve text and full-search fallback. Native requests need native fixtures [41, 42].
Local reuse and invalidation	Offline, empty collection, imports, rename/removal, sessions/tags changed and large collections. Rebuild relevant indexes only; never send local searches/notes to a store.
Appearance and measurement	Existing themes and any future Comfort variants; contrast, focus, popup clipping/scrolling. Record typing-to-visible time, provider calls, details fetched only when needed and selection accuracy; use measured budgets, not a promised speedup.

Current allocation: R31-R32 are now explicitly promoted. The original review sequence below remains a useful conditional implementation order; its earlier promotion boundary is superseded by this owner request.

Suggested owner-review sequence: validate provider access and the accessible control; prototype Steam-only lightweight suggestions; verify exact-product selection; assess Epic/GOG and All stores; then reuse local adapters where they improve task completion. Technical feasibility does not authorize implementation or roadmap promotion. No hosting is required by the proposed local/native design; any later paid dependency or payment commitment still requires the owner's personal choice.

Five themes: design rationale

Five base themes plus one optional Comfort modifier, not six dropdown themes. These proposals may replace existing palettes. The names and sample colors below are recommendations for a future prototype, not shipped changes.

Most used cannot be established from the sources reviewed: product help pages describe available choices, not how many people select them. Dark variants and light alternatives are well represented in the examples below. Familiarity supports a dark gaming default; it does not prove that dark is healthier or best for every user.

Reference	What is established	Implication and limits
Discord [49]	Light plus Ash, Dark and Onyx; Appearance behind the Settings cog.	Offer distinct light/dark choices and keep controls in Settings. Adjacent gaming/social app; no selection shares published here.
Playnite [50]	Custom themes; Settings > Appearance > General uses a Theme dropdown.	Keep selection discoverable. Community customization is an option, not evidence of a most-used palette.
Steam [51]	Valve's 2023 desktop update coordinated dialogs, menus, fonts and colors.	Polish the whole interface consistently. This dated design reference does not establish current theme adoption.
GOG [52]	Its website exposes System, Dark and Light color modes; Galaxy emphasizes custom library views/artwork.	Allow preference and lighting differences. Website modes must not be presented as verified Galaxy client theme options.

Recommend three dark and two light themes. This broadens the current four-dark/one-light set. Keep familiar names where useful; replace Cozy with Paper only after preference migration is designed. A warm dark appearance remains possible through Dark + Comfort. Color mood descriptions below are aesthetic intent, not established medical effects.

Theme	Direction	Purpose and polish
Gaming	Keep and polish; navy/cyan	Default identity and cover-rich browsing. Use cyan for actions/focus, simplify hero glow, align surfaces and strengthen text hierarchy.
Dark	Replace lavender-heavy styling with neutral charcoal	A restrained alternative for dim rooms and users who prefer fewer bright colors. Preserve clear separation between panels and controls.

Theme	Direction	Purpose and polish
Light	Keep; neutral light/blue	Bright-room use and longer notes or information reading. Avoid thin gray text and bright full-screen transitions from a dark theme.
Paper	Replace Cozy; warm light/ochre	A second light choice with a softer off-white appearance. It is a preference alternative, not a proven healthier color temperature.
Calm	Keep; muted green/gray	Quiet browsing with little decoration. Retain existing low-motion behavior and original artwork; add Comfort as a further optional adjustment.

Shared polish: consistent readable type/spacing, stable card sizes, unclipped titles, restrained accents, visible selected/hover/focus states and clear progress/error messages. Avoid full-screen glow, translucent text over unpredictable artwork and automatic video. Keep covers in their original colors. Preview the library, a long note and an error in Settings before finalizing a theme.

Comfort and blue-light evidence

Comfort is an optional visual preference; the evidence does not validate a GameVault palette as medical protection or a guaranteed sleep intervention.

Evidence	Finding and scope	Design consequence
Eye discomfort [53]	AAO links temporary screen discomfort to factors such as reduced blinking, glare and prolonged use; screen blue light has not been shown to damage eyes.	Offer readable text and low visual intensity. Recommend suitable screen brightness/room lighting and breaks in optional help; never call the checkbox eye protection.
Reading polarity [54]	The 2013 study recruited 169 younger/older adults (two later excluded); dark text on a light display improved tested acuity/proofreading.	Keep Light and Paper light with Comfort on. Controlled reading tasks and vision exclusions limit generalization; this is not an app comfort trial.
Evening light [56]	A small laboratory crossover study found sleep/circadian differences between evening light-emitting eReader and printed-book use.	Evening exposure deserves attention, but this comparison did not test GameVault themes or prove a health benefit from any proposed palette.
Warm filter limits [55]	A randomized study of 167 adults aged 18-24 compared Night Shift on/off and no phone for seven nights; full-sample sleep outcomes did not differ significantly.	Warm tint alone cannot be sold as reliable sleep protection. The age range, duration and phone setting also limit extrapolation to a desktop app.

Blue light: Screen blue light has not been shown to damage human eyes; that differs from evening light affecting the body clock. A warm palette can change some interface colors, but cannot guarantee reduced exposure across covers, other windows, display brightness or hardware. Color warmth alone is not reliable sleep protection. Preserve personal choice and describe Comfort as a preference; do not claim a blue-light filter or a medically safe theme [53, 55, 56].

Shared Comfort requirements

- Tailor each palette: soften large surface colors and accent saturation while retaining the selected theme's identity and dark/light polarity. Prefer solid text backgrounds; avoid a blanket opacity, sepia or blur filter.
- Keep meaningful text at least 4.5:1, large text at least 3:1, and required control boundaries/state icons at least 3:1 against adjacent colors. Aim for 7:1 body text where practical; measure real states, not swatches alone [57, 58].
- Remove decorative glow, card lift, parallax, animated gradients and unnecessary transitions; use calm progress indicators without flashing. Honor OS reduced motion even when Comfort is off [59].
- Retain readable font weight, generous line spacing, text scaling, keyboard focus and labels alongside status colors. Comfort must not override Windows contrast themes or assistive/user display preferences.
- Preserve artwork and achievement image colors by default. Reduce surrounding decoration instead. An app palette does not control display backlight, physical brightness or the spectrum of all on-screen content.
- Provide optional help on glare, suitable brightness, blinking/breaks and limiting evening screen use [53]. These are separate habits; switching this checkbox cannot guarantee eye health, reduced fatigue or better sleep.

Tailored Comfort variants and state

Prototype colors only. Ratios below are calculated from these opaque sRGB pairs using the WCAG formula; body/secondary text use the lowest ratio against background or panel, and button text is checked against its accent. They validate sample pairs, not the complete app or a health benefit.

Gaming / Base

Library - Notes - Progress

Play

Body 13.2:1 / Secondary 8.0:1 / Button 8.6:1

Gaming / Comfort

Library - Notes - Progress

Play

Body 11.6:1 / Secondary 8.0:1 / Button 7.6:1

Gaming: Lift near-black navy to softer slate; replace luminous cyan with muted blue/cyan; remove hero glow. Keep the gaming identity.

Dark / Base

Library - Notes - Progress

Play

Body 12.9:1 / Secondary 7.7:1 / Button 7.7:1

Dark / Comfort

Library - Notes - Progress

Play

Body 10.8:1 / Secondary 7.5:1 / Button 7.4:1

Dark: Use slightly warmer charcoal, cream-gray text and a subdued olive-gray accent. Avoid white-on-pure-black extremes; preserve contrast.

Light / Base

Library - Notes - Progress

Play

Body 13.2:1 / Secondary 5.5:1 / Button 6.9:1

Light / Comfort

Library - Notes - Progress

Play

Body 11.2:1 / Secondary 5.0:1 / Button 5.6:1

Light: Use neutral off-white surfaces and quieter blue accents. Keep dark text and light polarity; do not turn the theme dark.

Paper / Base

Library - Notes - Progress

Play

Body 11.3:1 / Secondary 5.1:1 / Button 5.7:1

Paper / Comfort

Library - Notes - Progress

Play

Body 10.1:1 / Secondary 4.8:1 / Button 5.3:1

Paper: Reduce yellow/ochre intensity; use restrained warm-gray/ivory surfaces. Keep strong dark text rather than dimming the whole page.

Calm / Base

Library - Notes - Progress

Play

Body 10.4:1 / Secondary 7.5:1 / Button 7.6:1

Calm / Comfort

Library - Notes - Progress

Play

Body 9.3:1 / Secondary 6.8:1 / Button 7.1:1

Calm: Lift deepest green surfaces slightly, mute sage accents and remove remaining decoration. Keep panel separation; avoid faint green-on-green controls.

Settings controls: Settings > Appearance: Theme [Gaming / Dark / Light / Paper / Calm], then [] Comfort mode. Helper text: Reduce visual intensity and motion while keeping your selected theme. Put a live miniature preview beside these controls. Keep the dropdown inside the in-app Settings panel and remove the duplicate main-toolbar selector in the future implementation.

Saved state: Persist one base-theme choice and one independent Comfort on/off preference. Comfort stays on when themes change; off restores the base palette. New installs and older backups without the setting default to off. Restart and JSON backup/restore preserve both. Theme switching must not reload the view, discard edits or cancel sync; show a failed save and restore the last saved choice.

Cozy migration: If Cozy is retired, keep old files readable. Offer the owner a choice of Paper or Dark + Comfort, or explicitly record a one-time migration. Preserve the previous appearance until a replacement is confirmed; do not silently replace a saved warm dark preference with a light screen.

Validation	Required check
Readability	Check all 10 combinations on library, notes, wishlist, achievements, dialogs and progress/errors. Include hover, selected, keyboard focus, placeholders and real artwork backgrounds.
Accessibility	Keyboard/screen-reader names for dropdown and checkbox; 200% text, mixed DPI and small/ultrawide windows. Test OS reduced motion and Windows contrast themes; no color-only status or flashing.
State and workflows	Switch while editing a note or while sync runs; close/reopen Settings. Verify no lost draft, canceled job or duplicate task. Test restart, old/new backups and rejected preference saves.
Comfort evaluation	Compare theme pairs with the same tasks in dim and bright rooms on LCD/OLED displays. Record reading errors, task time and individual comfort preferences; a small user check establishes usability, not medical efficacy.

Future implementation sequence: agree the portfolio and Cozy migration; prototype shared appearance rules and Settings controls; build the five tailored Comfort palettes; validate all ten combinations and state/workflow checks; record evidence before marking related roadmap work complete. Paid design/research services require the owner's decision first.

Recurring verification matrix

This matrix is proposed testing scope, not a list of checks newly run or failures proven in 0.3.8. Use browser fixtures for UI behavior, native fixtures for Windows storage/provider boundaries, and the installed app for window lifetime and restart. Playwright browser network mocks do not automatically intercept Rust/native HTTPS requests [41, 42, 43]. Isolate database, credentials, ports and WebView2 profile per test; use fixture credentials and exported sample collections, not the owner's live collection [42]. Keep debug automation ports confined to test builds. Use deterministic response gates/events rather than guessing a sleep duration.

Area and cadence	Scenario and required outcome
Settings and job lifetime / After sync/UI changes	Start each supported library/artwork workflow; close/reopen Settings, switch views and minimize. Continue once, keep progress visible, preserve edits and pending review.
Cancellation and stale replies / After task/provider changes	Cancel during each stage, change query/account/region, then deliver old replies. No late commit; completed results retained; retry only unresolved work.
Repeated or competing actions / After shared-state changes	Double-click Sync, start covers/achievements together, edit a game during a delayed save. Prevent duplicate jobs and stale overwrites; show clear status.
Provider and network failures / Provider changes + milestones	Timeout, offline, rate limit, private profile, invalid/partial payload, one store down. Preserve cached data and unaffected results; explain retry/availability.
Restart, crash and sleep / Lifecycle changes + releases	Exit/crash between scan, import, artwork and save; restart or sleep/resume. Restore committed work, report unfinished steps, avoid duplicate sessions/imports.
Storage and backup safety / Storage changes + releases	Corrupt input, disk full, denied write, conflicting save, oversized artwork and migration. Retain prior collection; preview restore and prove old-snapshot recovery.
Identity and metadata / Import/metadata changes	Same title across stores/editions, DLC, renamed game, ambiguous match and manual override. No wrong merge; curated fields and supported cover credits survive.
Playtime and achievements / Tracking/progress changes	Delayed Steam totals, edited sessions, overlapping timers, account switch, missing rarity and hidden achievements. Correct totals; preserve progress and spoiler rules.
Wishlist and notifications / Price/alert changes	Region/currency fallback, stale price, different editions, duplicate offers, thresholds and rule restore. Correct product/price context; no duplicate or unwanted alert.
Accessibility and layout / New screens + milestones	Keyboard-only, screen reader, focus return, high DPI/200% text, small/ultrawide windows and languages. Reach all controls; announce errors; preserve user text.
Themes and Comfort / Theme changes + milestones	All five themes with Comfort off/on: text/control contrast, focus, scaling, Windows contrast themes, reduced motion, restart/backup and failed saves. Switching preserves edits and active sync.
Scale and long sessions / Performance changes + milestones	Measure realistic and 1k/10k/50k metadata fixtures separately from near-25-MiB artwork capacity. Record latency, DOM count, CPU, total memory, save stages and lock-wait responsiveness using the code-study protocol; retain first-run failures.
Installer, update and security / Every release; accounts when built	Clean normal-user install, repair/uninstall, supported Windows versions, altered package, interrupted update, secret redaction and future account recovery. Data retained; invalid updates rejected.
Provider field capabilities / Connector changes + milestones	Unsupported favorites, install without ownership, missing DLC base, editions and unavailable Last Played. Show accurate source/limits; preserve curated fields and PC-only eligibility.
Launch and automatic detection / Tracking/launcher changes	Normal-user launch, external launch, child-process handoff, duplicate names/PID reuse, permission denial, crash, sleep and restart. One correct recoverable session; no fabricated playtime.
Valuation and discovery quality / Price/recommendation changes	Mixed currencies, unknown/free costs, gifts/refunds/bundles, partial valuation; seeded recommendations, cold start, hidden games and repeated draws. Honest totals, useful varied eligible picks.
Onboarding and delivery consent / Setup/notification changes	Local-only first run, skipped/failed steps, restart/upgrade/restore; denied alerts, unsubscribe, quiet hours and repeated delivery. Preserve choices and show only wanted notifications.
Cross-device API and mobile / Before mobile/sync; after changes	Two offline devices, conflicting notes/deletion/sessions, repeated mutations, foreign-account access, revoked token and older schema. No lost unrelated data; adaptive touch layout and honest device capabilities.

After each affected change, run the relevant focused cases and permanent regressions. At a phase milestone, run the full workflow matrix and manual accessibility checks. Before release, add the installed-app, clean-install/upgrade and wider Windows checks, with separate hands-on provider verification. Record intermittent and first-run failures, exact source/build and accepted gaps; keep preview and production totals separate. No account service, commercial API automation or paid request is authorized by this plan. The added roadmap-gap cases are proposed until the corresponding features exist; evaluate them with isolated fixture data and a supported-platform matrix. A documented unsupported provider field is not a zero value or a failed local collection.

Research-to-roadmap coverage

All earlier R01-R32 and the newly researched R33-R41 are incorporated. Each topic appears once under its lead phase; the item links below identify cross-phase obligations. Completion also requires applicable delivery details and regression checks. Existing item numbers are preserved.

Research requirement	Phase	Related numbered items
R01 Backup history and tested restoration	1	1, 31, 32, 38
R02 Connector capabilities and honest connection status	2	4, 5, 6, 7, 8
R03 Offline safety before cross-device sync	11	2, 33, 35
R04 Repairable matches and protected metadata	2	5, 7, 15, 16
R05 Saved filters, bulk edits and safe custom launch targets	4	10, 12, 13, 14
R06 Optional accounts, recovery and trusted devices	1	2, 36, 37
R07 Privacy controls and account-data lifecycle	1	2, 29, 33, 35, 39, 42
R08 Extend protected credentials to future sign-in	10	4, 31, 36, 37
R09 Publisher signing and update recovery	10	32, 41
R10 Feedback, Bug Report and reviewable diagnostics	4	39, 42
R11 Extend the existing release gate and maintenance record	10	38, 41, 43, 46
R12 Accessibility and display scaling across the app	1	1, 14, 44, 47, 48
R13 Measured large-library performance and storage limits	1	1, 5, 13, 15, 38
R14 Explainable playtime and resilient session tracking	3	8, 9, 10, 11
R15 Price provenance, editions and reliable comparisons	7	21, 22, 23, 27, 28
R16 Useful alerts and controllable recommendations	8	24, 25, 26, 28, 29
R17 Achievement coverage and refresh transparency	6	18, 19, 20
R18 One source for release and setup guidance	10	30, 31, 38, 41
R19 In-app Settings and independent background work	2	4, 5, 44, 45
R20 Regression checks for real user interruptions	1	38, 43, 46
R21 Five purposeful themes and a polished default	1	47
R22 Comfort as a tailored option on every theme	1	48
R23 Render a bounded library and reuse search work	4	1, 12, 13, 14, 38
R24 Update progress without rebuilding recommendations	4	14, 24, 25, 26, 38, 45
R25 Reduce full-collection copying and repeated saves	1	2, 5, 38, 45
R26 Keep database work off the native UI thread	1	1, 2, 38, 45
R27 Separate artwork files from collection metadata	5	15, 16, 17, 38
R28 Reuse network connections and avoid duplicate lookups	2	4, 5, 18, 21, 38
R29 Cancel resource work and reject late results consistently	2	5, 15, 18, 44, 45, 46
R30 Make the source easier to review without a framework rewrite	1	2, 38
R31 Wishlist search suggestions while typing	9	27, 28
R32 Reuse suggestions for local GameVault searches	4	12, 14, 18, 27
R33 Provider feasibility: platforms, favorites, DLC and Last Played	2	3, 4, 5, 6, 7, 8
R34 Safe game launching and automatic session detection	3	9, 10, 11
R35 Collection layout, categories and Settings interaction	4	1, 12, 13, 14, 44
R36 Achievement identity, private states and completion denominators	6	18, 19, 20
R37 Reliable purchase records and honest library valuations	7	21, 22, 23
R38 Evaluate recommendation usefulness, variety and repeat control	8	24, 25, 26
R39 Closed-app alerts, delivery preferences and bounded monitoring	9	2, 28, 29, 40
R40 Resumable first-run setup without forced accounts or libraries	10	30, 31, 36
R41 Mobile contracts, conflict-safe sync and device-specific capabilities	11	2, 3, 32, 33, 34, 35

Evidence sources

Citations open this numbered index; titles link to original sources. Private project links may require access. Earlier baselines remain historical. Sources 91-111 support the remaining-roadmap review; guidance checked 5 October 2026. Recheck provider terms/access/costs before implementation.

- [01] Playnite - Backup and restore
- [02] SQLite - Online Backup API
- [03] Steamworks - IPlayerService / GetOwnedGames
- [04] Playnite - Game metadata
- [05] Playnite - Filters and filter presets
- [06] OWASP - Session Management
- [07] OWASP - Multifactor Authentication
- [08] NIST SP 800-63B - Authentication
- [09] OWASP - Forgot Password
- [10] European Commission - Privacy obligations
- [11] RFC 8252 - OAuth 2.0 for Native Apps
- [12] Microsoft - CryptProtectData scope and limits
- [13] Microsoft - SmartScreen reputation
- [14] OWASP - Logging and sensitive-data exclusions
- [15] NIST SP 800-218 - Secure development (SSDF)
- [16] Microsoft - Develop accessible Windows apps
- [17] W3C - CSS containment and off-screen rendering
- [18] RFC 3339 - Internet timestamps
- [19] IsThereAnyDeal - API features and terms
- [20] VirusTotal - How it works and file sharing
- [21] VirusTotal - False positives and vendor verdicts
- [22] VirusTotal - Public vs Premium API restrictions
- [23] GameVault - published 0.3.8 release
- [24] GameVault - user guide and limits
- [25] GameVault - transactional storage and recovery
- [26] GameVault - conservative library matching
- [27] GameVault - local recommendation rules
- [28] GameVault - Windows Credential Manager
- [29] GameVault - verified updater and snapshot
- [30] GameVault - store search and regional offers
- [31] GameVault - collection, filters and session UI
- [32] GameVault - achievements and API limits
- [33] GameVault - 0.3.8 production evidence
- [34] GameVault - SteamGridDB-only covers
- [35] GitHub - successful 0.3.8 production run
- [36] Microsoft - Windows credential writing
- [37] W3C - WCAG 2.2 accessibility criteria
- [38] GameVault - 500-game/20,000-session checks
- [39] GameVault - Steam playtime reconciliation
- [40] GameVault - Settings window and task ownership
- [41] Playwright - network and failure simulation
- [42] Playwright - WebView2 automation and isolation
- [43] Tauri - application testing layers
- [44] Playwright - retries and intermittent results
- [45] Playwright - accessibility testing limitations
- [46] GameVault - released theme names and persistence
- [47] GameVault - released palette and motion rules
- [48] GameVault - existing Calm research and limits
- [49] Discord - default themes and Appearance settings
- [50] Playnite - Settings theme dropdown and custom themes
- [51] Valve - Steam desktop visual refresh (2023)
- [52] GOG - Galaxy page and website color-mode choices
- [53] AAO - Digital Devices and Your Eyes
- [54] Piepenbrock et al. (2013) - display polarity
- [55] Duraccio et al. (2021) - randomized Night Shift trial
- [56] Chang et al. (2015) - evening eReader crossover study

Evidence sources continued

Citations open this numbered index; titles link to original sources. Private project links may require access. Earlier baselines remain historical. Sources 91-111 support the remaining-roadmap review; guidance checked 5 October 2026. Recheck provider terms/access/costs before implementation.

- [57] W3C - minimum text contrast
- [58] W3C - non-text contrast for controls and graphics
- [59] W3C - respect the reduced-motion preference
- [60] GameVault - library rendering, indexes and saves
- [61] GameVault - progress redraws and icon batches
- [62] GameVault - cover resizing and per-game saves
- [63] GameVault - synchronous SQLite snapshot commands
- [64] GameVault - full-snapshot window reloads
- [65] GameVault - HTTP client creation and response bounds
- [66] GameVault - provider requests and catalog cancellation
- [67] GameVault - performance fixture assertions
- [68] GameVault - project layout and generated preview
- [69] request - reuse a Client and its connection pool
- [70] Tauri - command execution and async work
- [71] Tauri - targeted events and streaming channels
- [72] Chrome DevTools - runtime performance recording
- [73] MDN - Performance marks and measurements
- [74] SQLite - transactions and writer concurrency
- [75] Tokio - cancellation and graceful shutdown
- [76] Tokio - shared state and dedicated ownership
- [77] GameVault - published 0.3.9 release
- [78] GameVault - Wishlist input and stale-reply checks
- [79] GameVault - store queries and product validation
- [80] W3C APG - combobox interaction pattern
- [81] W3C APG - manual-selection autocomplete example
- [82] W3C WAI-ARIA 1.2 - combobox semantics
- [83] MDN - datalist accessibility limitations
- [84] Valve - IStoreService paged app catalog
- [85] Valve - deprecated ISteamApps app catalog
- [86] Epic - scope of product ecommerce APIs
- [87] MDN - text composition completion
- [88] MDN - aborting browser fetch requests
- [89] MDN - locale-sensitive text comparison
- [90] GameVault - retained 0.3.9 production evidence
- [91] Valve - installed apps and DLC checks: SDK scope
- [92] Valve - DLC identifiers and base-game relationships
- [93] Microsoft - process start events
- [94] Microsoft - process stop events
- [95] Microsoft - process identity, creation time and PID reuse
- [96] Tauri - scoped paths and protocol opening
- [97] Valve - achievement schemas, player progress and global rates
- [98] ECB - dated reference exchange rates and limitations
- [99] Recommenders - diversity, coverage and relevance evaluation
- [100] Android - offline data, queues and conflict resolution
- [101] Android - persistent work and scheduling limits
- [102] Android - contextual notification permission and denial
- [103] Apple - concise, skippable and contextual onboarding
- [104] W3C APG - modal dialog focus and dismissal
- [105] RFC 9110 - HTTP preconditions and idempotent operations
- [106] RFC 6585 - rate-limit responses and Retry-After
- [107] RFC 9457 - structured API failures and safe diagnostics
- [108] Android - adaptive layouts as window size changes
- [109] Android - limits on visibility of other installed apps
- [110] rust_decimal - exact decimal representation for money
- [111] OWASP - HTTPS, endpoint authorization and API safeguards

GAMEVAULT · One library. Every game. Your entire gaming history.